

Завдання №2: Реалізація алгоритму RLE

1 Мета роботи

Опанувати поняття коду, дослідити різні види перетворень даних, представлених двійковими словами (байтами).

2 Завдання роботи

Вам необхідно реалізувати алгоритм стискання даних методом RLE.

3 Алгоритм стискання RLE

3.1 Загальні положення

Алгоритм RLE (від англ. *Run Length Encoding*) — це алгоритм, який стискає дані шляхом заміни послідовностей однакових байтів на спеціальні записи виду «(довжина послідовності, значення байту)».

Вхідний потік байтів кодується таким чином:

- послідовність з ℓ однакових байтів “a, a, ..., a” (де $2 \leq \ell \leq 129$) замінюється на послідовність з двох байтів “L, a”, де $L = 128 + \ell - 2$; іншими словами, байт L у двійковому записі має вид $1\|\text{xxxxxxx}$, де xxxxxxx — семибітове двійкове зображення числа $\ell - 2$;
- послідовність з ℓ різних байтів “a, b, ..., z” (де $1 \leq \ell \leq 128$) замінюється на послідовність з байтів “L, a, b, ..., z”, де $L = \ell - 1$; іншими словами, байт L у двійковому записі має вид $0\|\text{xxxxxxx}$, де xxxxxxx — семибітове двійкове зображення числа $\ell - 1$.

3.2 Вимоги до кодера

RLE-кодер (програма стискання даних) повинен бути реалізований як окрема програма або як модуль в межах більш функціональної програми, який викликається спеціальною командою (з інтерфейсу програми, з меню чи з командного рядку консолі).

1. Кодер повинен приймати на вхід ім'я файла для кодування та (опціонально) ім'я файла для зберігання закодованого результату.
2. Якщо користувач не надав ім'я файла для зберігання закодованого результату, кодер повинен створити такий файл сам, додавши до імені оригінального файлу розширення ‘.rle’.
3. Вхідний файл повинен трактуватись як файл двійкових даних (потік байтів).

4. Стиснені дані повинні записуватись у вихідний файл як у файл двійкових даних (потік байтів).
5. Кодер може вставляти у вихідний файл довільні необхідні йому метадані (наприклад, заголовки чи якісь службові команди, які ви передбачаєте своєю реалізацією).

Алгоритм RLE не є однозначним в тому сенсі, що заданим вхідним даним у багатьох випадках може відповідати декілька стиснених образів. У своїх реалізаціях ви можете передбачити деякі евристичні оптимізації, які покращують якість стиснення — наприклад, відслідковувати короткі серії однакових байтів, які погано стискаються, і замінювати їх на нестиснену послідовність.

3.3 Вимоги до декодера

RLE-декодер (програма, яка розпаковує дані в оригінальний нестиснений вид) повинен бути реалізований як окрема програма або як модуль в межах більш функціональної програми, який викликається спеціальною командою (з інтерфейсу програми, з меню чи з командного рядку консолі).

1. Декодер повинен приймати на вхід ім'я стисненого файлу та ім'я файла для зберігання розпакованого результату.
2. (Опціонально) Декодер може запропонувати користувачу варіант імені для декодованого файлу: наприклад, якщо закодований файл має розширення `'rle'` — відкинувши таке розширення; або, якщо закодований файл у метаданих зберігає ім'я оригінального файлу, — запропонувати збережене ім'я. Але, в будь-якому випадку, користувач повинен або сам зафіксувати ім'я файлу виходу, або свідомо погодитись із пропозицією такого імені.
3. Файли входу та виходу повинні трактуватись як файли двійкових даних (потоки байтів).
4. Якщо ваша реалізація передбачає якісь метадані у стиснутому файлі, декодер повинен перевірити їх коректність та, у разі їх некоректності, вивести відповідне повідомлення про помилку.
5. Якщо декодер виявив неправильну команду алгоритму стиснення у вхідному файлі (команда каже зчитати послідовність байтів, яка виходить за кінець файлу), декодер повинен вивести відповідне повідомлення про помилку.
6. У випадку виникнення помилки декодування декодер може як повністю зупинити свою роботу, так і спробувати декодувати те, що зможе. Вибір поведінки залишається на ваш розсуд.

4 Вимоги до звіту

Результати вашої роботи подаються у вигляді коротенького звіту, який повинен включати опис та особливості вашої реалізації заданого алгоритму, а також приклади кодування, декодування та декодування спотворених даних.

Алгоритм RLE застосовний для специфічних джерел даних. Для наочного демонстрування роботи алгоритму рекомендується використовувати графічні файли у форматі BMP, причому зображення повинні містити великі монотонні фрагменти, які кодуються послідовностями однакових пікселів.

Ваш програмний код традиційно повинен надаватись у вигляді посилання на відкритий GitHub-репозиторій.